PH331 : Computational Physics Lab

Chapter V

Some Advanced Problems

Atma Anand
SC12B156

Date: November 9, 2014 **Department**: Physical Sciences

1 Problem 1: Bound motion

We consider the 1-dimensional motion of a particle of mass m in a time independent potential $V(x)$. The fact that the energy E will be conserved allows us to integrate the equation of motion and obtain a solution in a closed form

$$t - C = \sqrt{\frac{m}{2}} \int_{-a}^{a} \frac{1}{\sqrt{E - V}}\, dx$$

where for the choice $C = 0$ the particle is at the position x_i at the time $t = 0$ and x refers to its position at any arbitrary time t. We consider a particular case where the particle is in bound motion between two points a and b where $V(a) = E$ and $V(b) = E$ and $V(x)E$ for $a \leq x \leq b$. The time period of the oscillation T is given by

$$T = 2\sqrt{\frac{m}{2}} \int_{-a}^{a} \frac{1}{\sqrt{E - V}}\, dx$$

First consider a particle with $m = 1Kg$ in the potential $V(x) = (1/2)\alpha x^2$ with $\alpha = 4Kgs$ 1. Numerically calculate the time period of oscillation and check this against the expected value. Verify that the frequency does not depend on the amplitude of oscillation. Next consider a potential $V(x) = exp((1/2)\alpha x^2)$ 2. Numerically verify that for small amplitude oscillations you recover the same results as the simple harmonic oscillator. The *float pot(float)*; time period is expected to be different for large amplitude oscillations. How does the time period vary with the amplitude of oscillations? Show this graphically.

1.1 Program Code:

```
#include <stdio.h>
#include <math.h>

double pot(double a) //double floating type for accuracy
{
    double E=0.0,al=4.0;
    double dx=a*1.0e-5, x=0.0, I=0.0;
    E=exp(0.5*al*a*a);
```

```
    dx= (dx>1e-5)?1e-5:dx;
    x=-a+dx/2.0;
    while(x<a)
    {
        I+= (1.0/sqrt(E - exp(0.5*al*x*x)))*dx;
        x+= dx;
    }
    return I;
}

int main()
{
    int i=0;
    double E=0.0,al=4.0;
    double a=1e-4, dx=1.0e-5, x=0.0, I=0.0;
    FILE *f;

    printf("\nTime Period for V=0.5*al*x*x: \n");
    printf("Amplitude\tTime Period: \n");
    for(i=0;i<=6;i++)
    {
        a*=10.0;
        x=0; I=0;
        dx=a/1e5;
        dx= (dx>1e-5)?1e-5:dx;
        E=0.5*al*a*a; x=-a+dx/2;
        while(x<a)
        {
            I+= (1.0/sqrt(E - 0.5*al*x*x))*dx;
            x+= dx;
        }
        printf("%G\t%G\n", a, sqrt(2.0)*I);
    }

    f=fopen("pot.txt","wb");
    printf("\nTime Period for V=exp(0.5*al*x*x):\n");
    printf("Amplitude\tTime Period: \n");
    a=1e-5; i=0;
    while(a<19)
    {
        I=sqrt(2.0)*pot(a);
        fprintf(f,"%G\t%G\n",a,I);
        if(i%5==0)
            printf("%G\t%G\n", a, I);
        i++;
        a*= (4*a>1)?1.1:2;
    }
    printf("%G\t%G\n", a/1.1, I);
    fclose(f);
    return 0;
}
```

1.2 Output:

```
[student@localhost SC12B156]$ ./prog1
Time Period for V=0.5*al*x*x:
Amplitude   Time Period:
0.001       3.13889
0.01        3.13889
0.1         3.13889
1           3.13889
10          3.14074
100         3.14132
1000        3.14146

Time Period for V=exp(0.5*al*x*x):
Amplitude   Time Period:
1E-05       3.1389
0.00032     3.13889
0.01024     3.13864
0.32768     2.89555
0.527732    2.54399
0.849918    1.8084
1.3688      0.715183
2.20447     0.0518691
3.55032     3.463E-05
5.71782     1.03454E-13
9.20861     3.89044E-36
14.8306     1.2658E-94
17.945      7.13861E-139
[student@localhost SC12B156]$
```

For very small integration interval ($a = 10^{-9}$), $t = 3.14157$ was obtained after a long computation time.

The variation of time period for the 2nd potential is shown graphically:

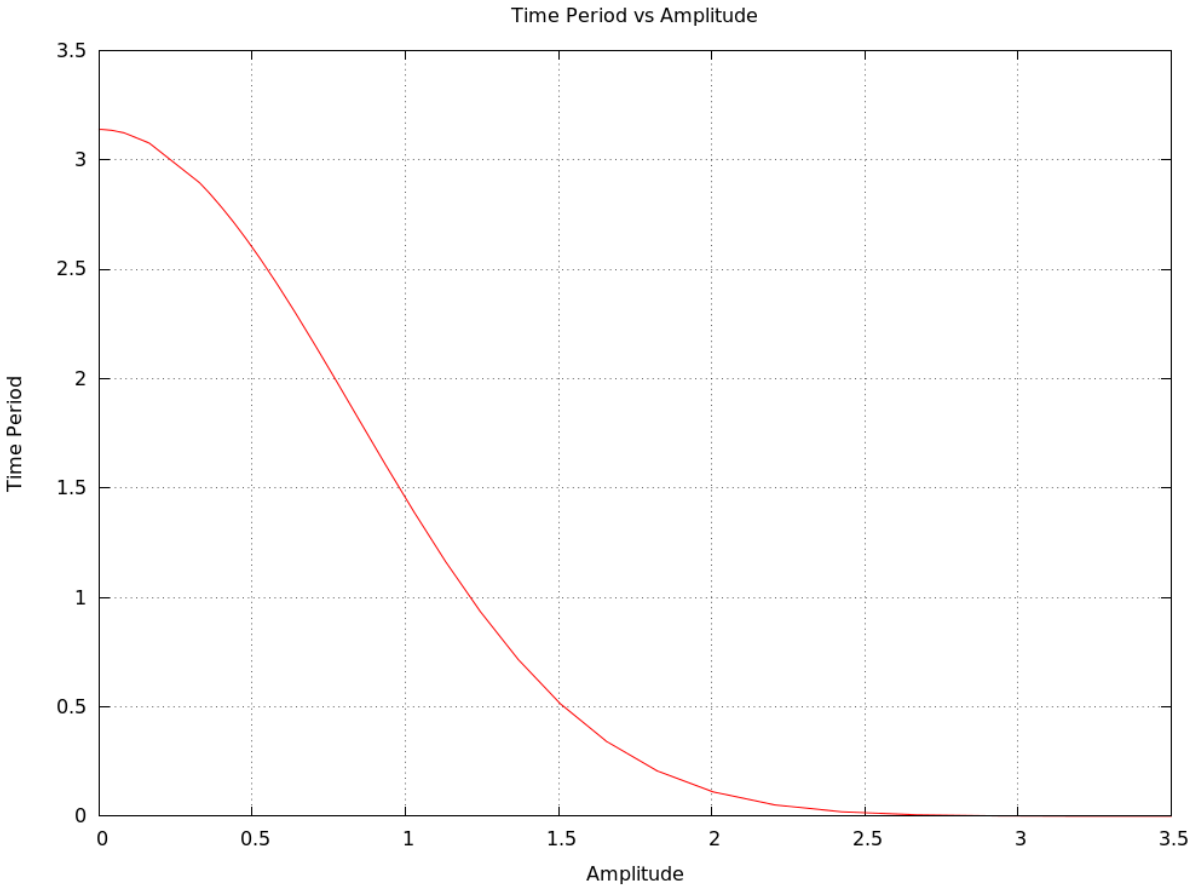


Figure 1: Trajectory for $x_0 = 0.5; y_0 = 0.0; v_{0,x} = 0.0; v_{0,y} = 1.63$

Following are the observations:

- The time period (T) for $V(x) = (1/2)\alpha x^2$ is approximately π.
- The time period for $V(x) = exp((1/2)\alpha x^2)$ is approximately π for very small amplitutes (a).
- For $V(x) = exp((1/2)\alpha x^2)$ the time period decreases exponentially (e^{-x^2}) with increase in amplitute of oscillation.

1.3 Discussion:

The integral given for potential $V(x) = (1/2)\alpha x^2$ is :

$$T = 2\sqrt{\frac{m}{2}} \int_{-a}^{a} \frac{1}{\sqrt{E - V}}\, dx$$

Substituting $E, V, \alpha = 4$ and $m = 1$, we get:

$$T = \int_{-a}^{a} \frac{1}{\sqrt{(a^2 - x^2)}}\, dx$$

This is the expression we used to calculate time period of particle computationally for first potential. The analytical solution of this integral is:

$$T = sin^{-1}\frac{x}{a} + C$$

Thus, we have:

$$T = sin^{-1}\frac{a}{a} - sin^{-1}\frac{-a}{a} = \frac{\pi}{2} - (-\frac{\pi}{2}) = \pi$$

Thus we see that the integral is **independent of amplitude**, which is what we have obtained.

For the second potential $V(x) = exp((1/2)\alpha x^2)$ the integral is :

$$T = \sqrt{2}\int_{-a}^{a} \frac{1}{\sqrt{exp((1/2)\alpha a^2) - exp((1/2)\alpha x^2)}}\, dx$$

This integral was approximately equal to π for very small amplitutes as:

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + ...$$

As $x \to 0$ we can neglect the higher terms and reduce it as: $exp(x) \approx 1 + x$
Thus our integral reduces to:

$$T \approx \sqrt{2}\int_{-a}^{a} \frac{1}{\sqrt{(1/2)\alpha a^2 - (1/2)\alpha x^2}}\, dx$$

Which is exactly the integral for $V(x) = (1/2)\alpha x^2$.

For higher values of a, the exponential function increases drastically and T $\to$ 0 for amplitudes beyond 2.5. The computation is in good agreement of the analytic solution.

2 Problem 2: Trajectory in an arbitrary potential

Write a program using Eulers Method and using the 2nd Order Runge-Kutta Method to find the phase space trajectory of a particle of unit mass in a 1-D potential V (x) = $\frac{1}{2}x^2$. a. Show the trajectory for the initial conditions (x, p) = (1, 0.1), (1, 0.1), (1, 10.). b. Check the conservation of energy for different step sizes, for both the Euler Method and the 2nd Order Runge Kutta Method.
Determine the step size where you have at least 1

2.1 Program Code:

```
#include <stdio.h>
#include <math.h>

void runge(double x, double p, char in[20])
{   // Differentiate V by 2nd Order Runge-Kutta Method to find force
    double E=0.0, dt=1.0e-5, t=0, tt=8.0, v=p;
    int i=0; // v = p as m=1
    FILE *f;
    f=fopen(in,"wb");
    while(t<tt)
    {
        E=0.5*x*x+0.5*v*v;
        if(i%50==0)
            fprintf(f,"%G\t%G\t%G\t%G\n",x,v,t,E);
        v+= -(x+v*dt/2)*dt; //-dV/dx = -x = F
```

```
        x+= v*dt;
        t+= dt;
        i=i%50+1;
    }
    fprintf(f,"%G\t%G\t%G\t%G\n",x,v,t,E);
    fclose(f);
}

void euler(double x, double p, char in[20])
{   // Differentiate V by Euler Method to find force
    double E=0.0, dt=1.0e-5, t=0, tt=8.0, v=p;
    int i=0; // v = p as m=1
    FILE *f;
    f=fopen(in,"wb");
    while(t<tt)
    {
        E=0.5*x*x+0.5*v*v;
        if(i%50==0)
            fprintf(f,"%G\t%G\t%G\t%G\n",x,v,t,E);
        v+= -x*dt; //-dV/dx = -x = F
        x+= v*dt;
        t+= dt;
        i=i%50+1;
    }
    fprintf(f,"%G\t%G\t%G\t%G\n",x,v,t,E);
    fclose(f);
}

int main()
{
    int i=4;
    double x,p;
    printf("Enter x and p: ");
    scanf("%lf%lf",&x,&p);
    runge(x, p, "rk.txt");
    euler(x, p, "er.txt");
    return 0;
}
```

2.2 Output:

```
[student@localhost SC12B156]$ ./prog2
Enter x and p: 1  0.1
[student@localhost SC12B156]$ ./prog2
Enter x and p: -1  0.1
[student@localhost SC12B156]$ ./prog2
Enter x and p: 1  10
[student@localhost SC12B156]$
```

Following are the output plots of phase space and energy vs time from gnuplot:

2.2.1 Euler Method $(|x|, p) = (1, 0.1)$

The plot for *x=1* and *x=-1* (or at any other $|x|$ value) for the *same p is the same*

Figure 2:

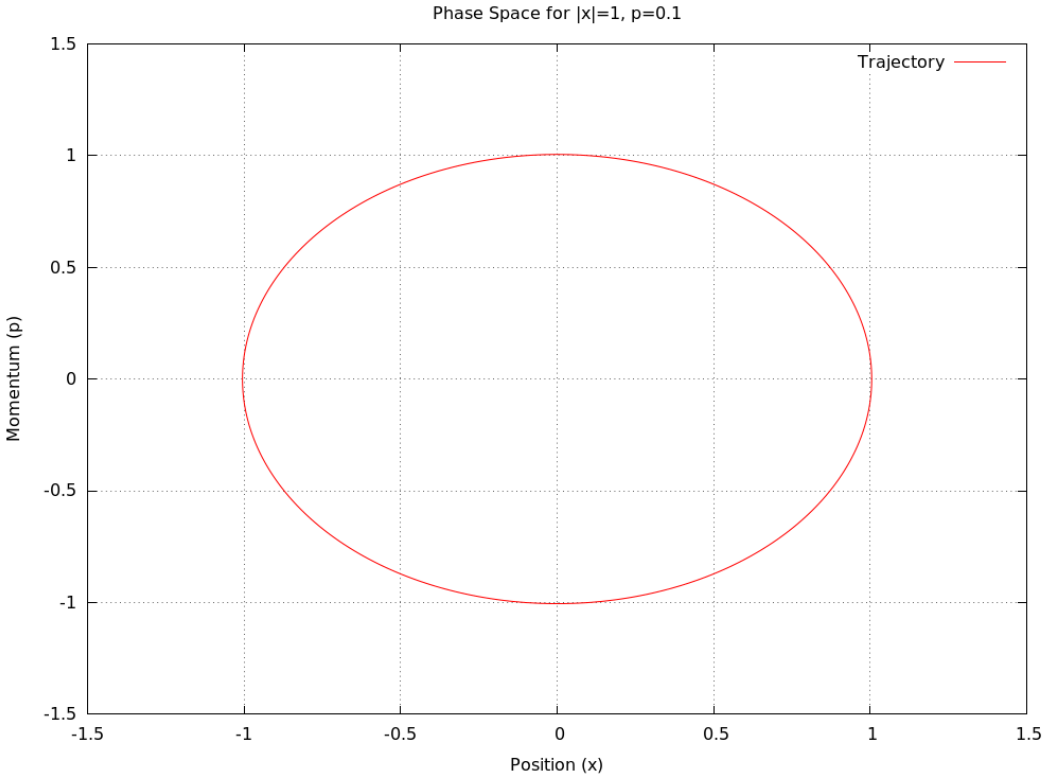
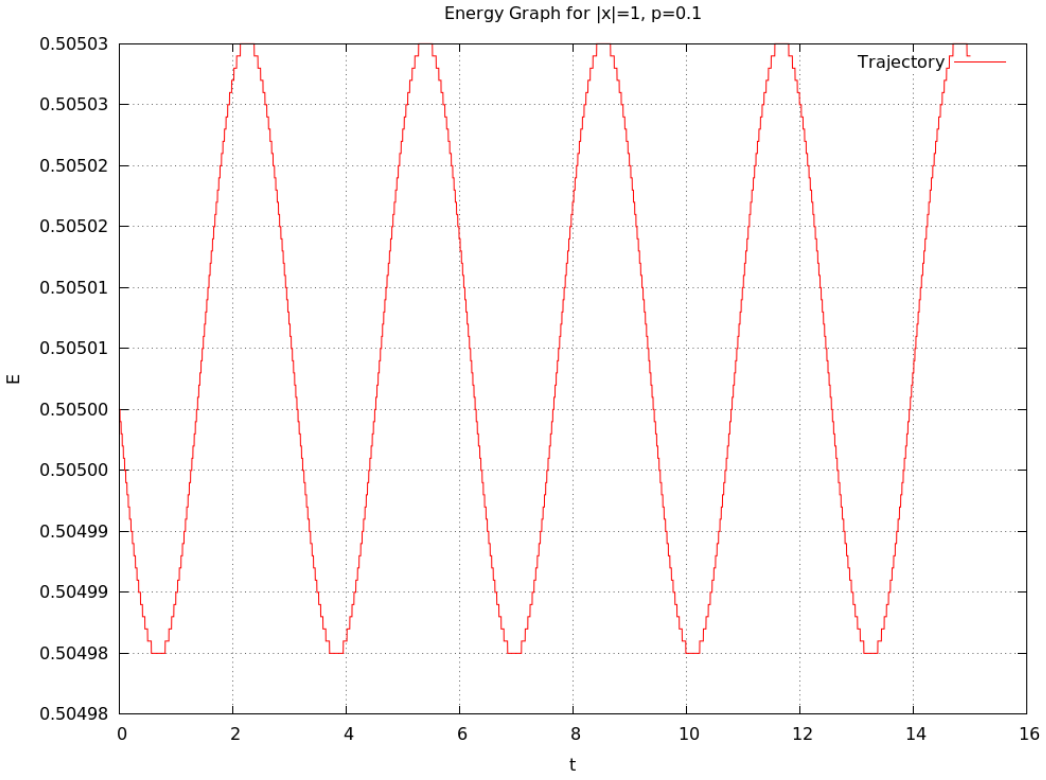


Figure 3:



2.2.2 Runge Kutta Method $(|x|, p) = (1, 0.1)$

Figure 4:

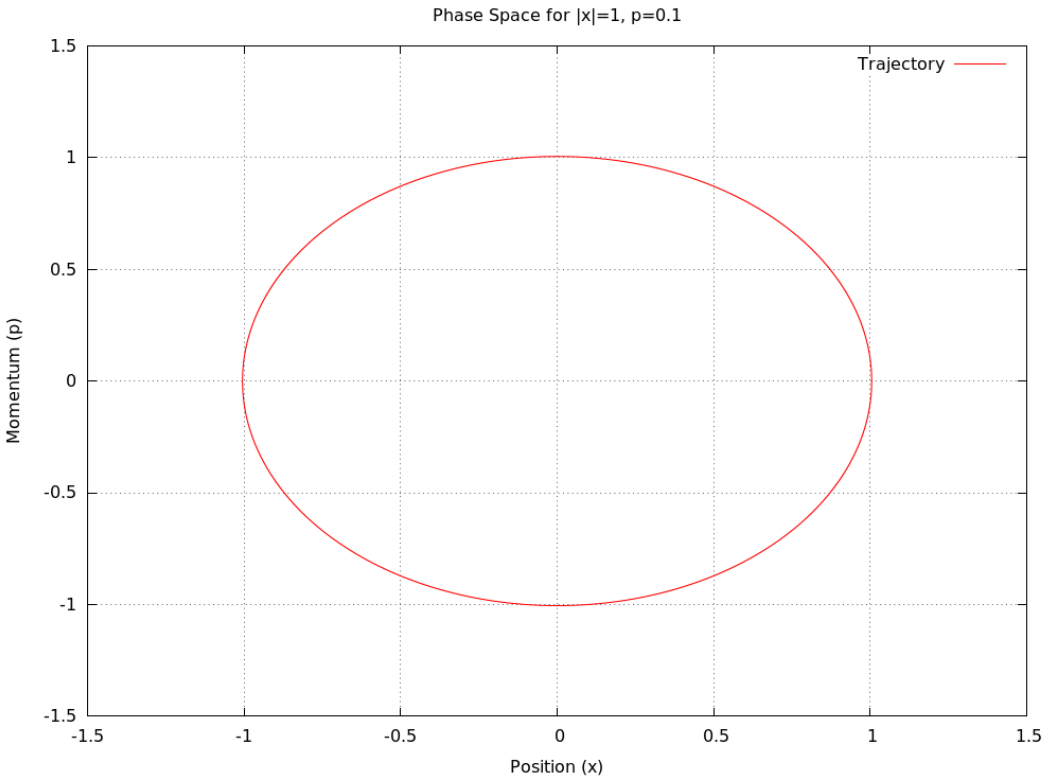
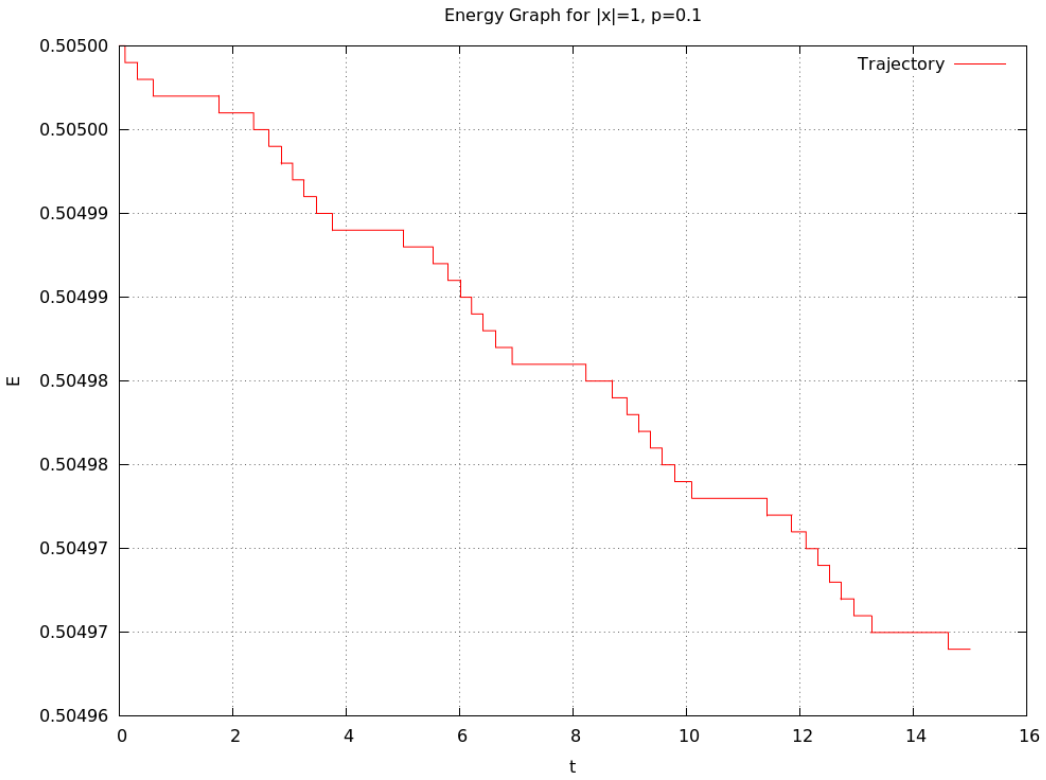


Figure 5:



2.2.3 Euler Method $(x, p) = (1, 10)$

Figure 6:

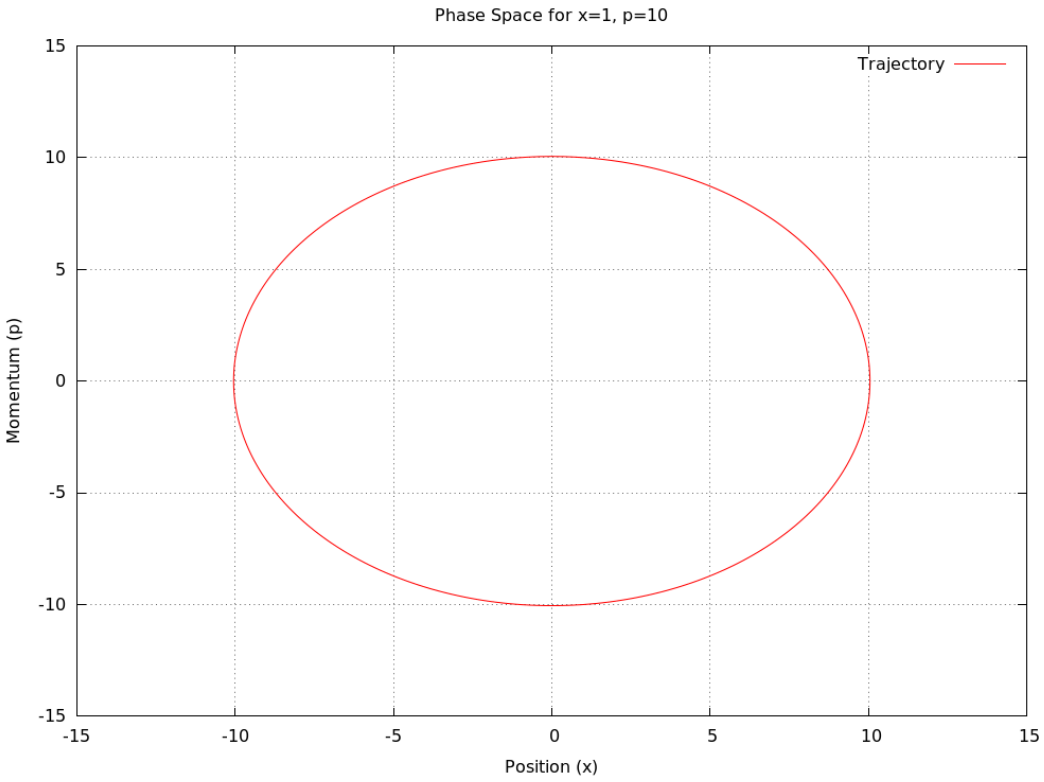
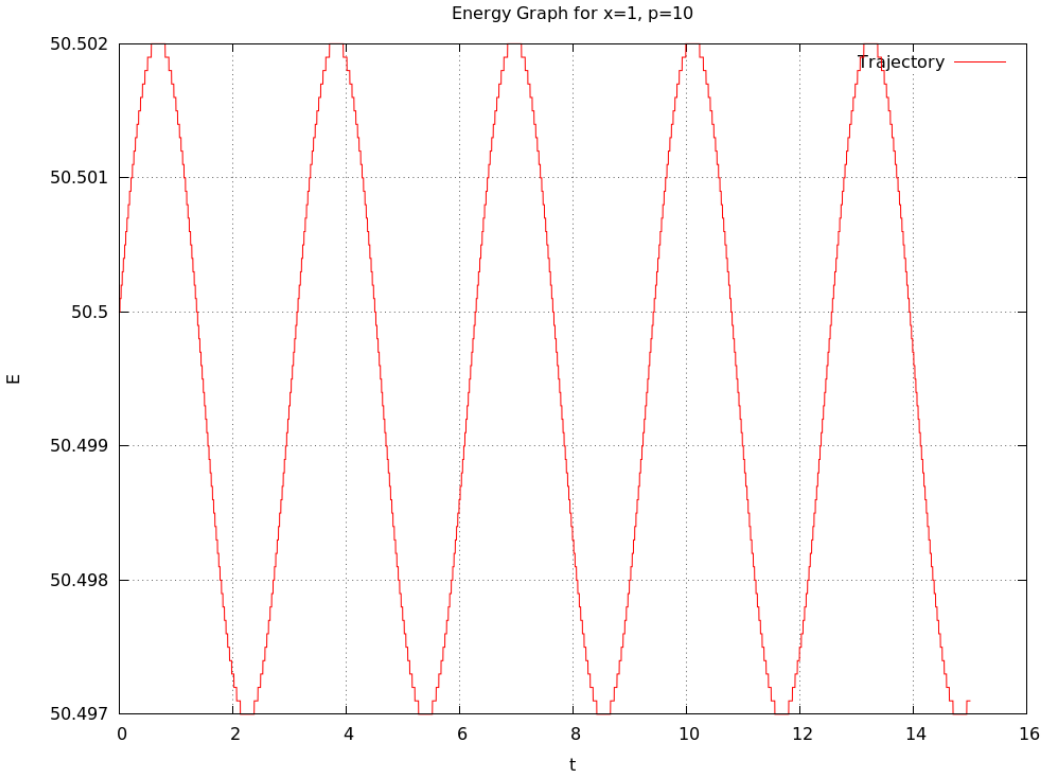


Figure 7:



2.2.4 Runge Kutta Method $(x, p) = (1, 10)$

Figure 8:

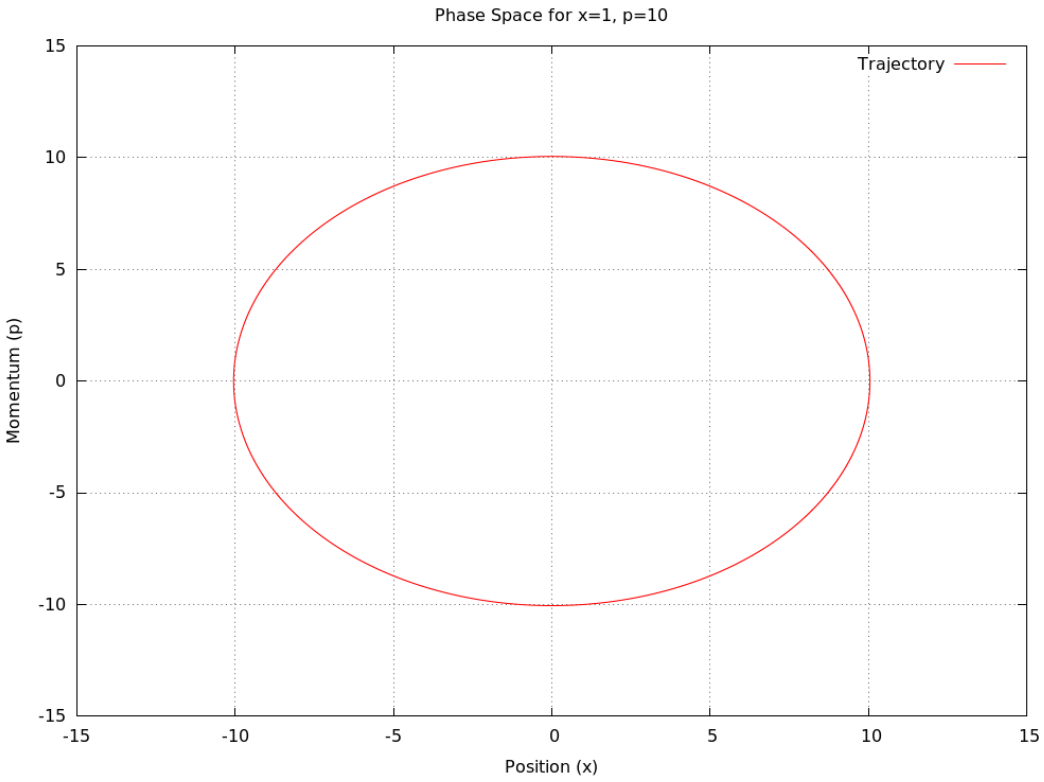
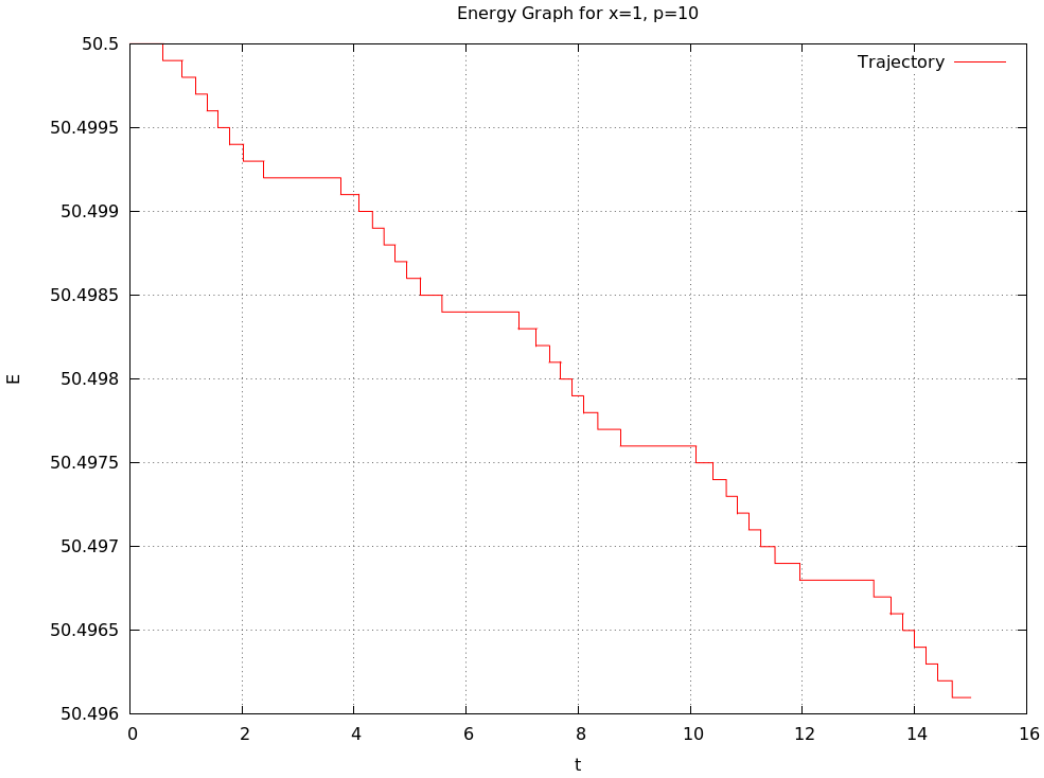


Figure 9:



Following are the observations:

- The phase space shows elliptical shape of position-momentum curve.

- Energy-Time plot is different for both methods - Varying sinusoidally in the Euler Method and decaying nearly as $x + |sin(x)|$ in Runge-Kutta Method.

2.3 Discussion

There exist various algorithms to compute differential equaions, each optimized for computation efficiency or a particular type of function. There is Euler's Method which is easy to implement but not very accurate as it considers only the 1st term of the function's Taylor Series Expansion. Second order Runge-Kutta Method also takes the 1st approx but at the midpoint of the interval. Simply put, in Euler method, to find the differential change in value of y with respect to x we use the curve's slope at starting point of dx (i.e. x_0) where as in 2nd order Runge-Kutta Method, we use the curve's slope at mid point of dx (i.e. $x_0 + dx/2$).
The Taylor Series Expansion is: $y(t+\delta t) = y(t) + y_t(t)\delta t + \frac{1}{2}y_{tt}\delta t^2 + \frac{1}{6}y_{ttt}\delta t^3$....

Also we notice change in energy over time due to quantization and truncation of values. Physically in real life there is no energy change. In Euler's Method the Energy lost in 1 part of the oscillation is recovered in the others (error+error=no error !). But in the Runge Kutta Method the energy decreases monotonically and over time the motion will dampen conputationally. The change is less when our step size (or time) is less. For ***Runge-Kutta there is no error for dt*** $\leq 10^{-7}$ *beyond which the energy decreases steadily in small steps. For* ***Euler's Method there is no error for dt*** $\leq 10^{-6}$ *beyond which the energy oscillates periodically in small steps. Thus although Runge Kutta is more effort taking, here it is inferior to the Euler's Method.*

3 Problem 3: Trajectory of a charged particle

Write a program to follow the motion of an electron in an electric field $E(x,t)$ and a magnetic field $B(x,t)$. Numerically determine the trajectory of an electron which starts at the origin with velocity $v = (1,1,1)m/sec$ for the following field configurations
a. Uniform magnetic field 10^{-4} T along the z axis.
b. Uniform magnetic field 10^{-6} T along the z axis and an uniform electric field $1V/m$ along the Y axis.

3.1 Program Code:

```
#include <stdio.h>
#include <math.h>

void path(double B_z, double E_y, char in[20])
{
    double v_x=1.0, v_y=1.0, v_z=1.0, t=0, dt=1e-9, tt=1e-5;
    double x=0.0,y=0.0,z=0.0, qm=-1.758E11;
    FILE *f; int i=0;
    if(E_y) tt=1.25e-4;
    f=fopen(in,"wb");
    while(t<tt)
    {
        if(i%10==0)
            fprintf(f,"%G\t%G\t%G\t%G\n",x,y,z,t);
        //Forces acting are Lorentz Force and Coulomb Force
```

```
        v_x= v_x + qm*B_z*v_y*dt;
        v_y= v_y - qm*B_z*v_x*dt + qm*E_y*dt;
        x= x + v_x*dt;
        y+= v_y*dt;
        z+= v_z*dt;
        t+= dt;
        ++i;
    }
    fprintf(f,"%G\t%G\t%G\t%G\n",x,y,z,t);
    fclose(f);
}

int main()
{
    path(1e-4, 0.0, "em_a.txt");
    path(1e-6, 1.0, "em_b.txt");
    return 0;
}
```

3.2 Output:

Following are the output plots of Electron trajectory for different E and B from 3D gnuplot:

3.2.1 Trajectory for $B_z = 10^{-4}$

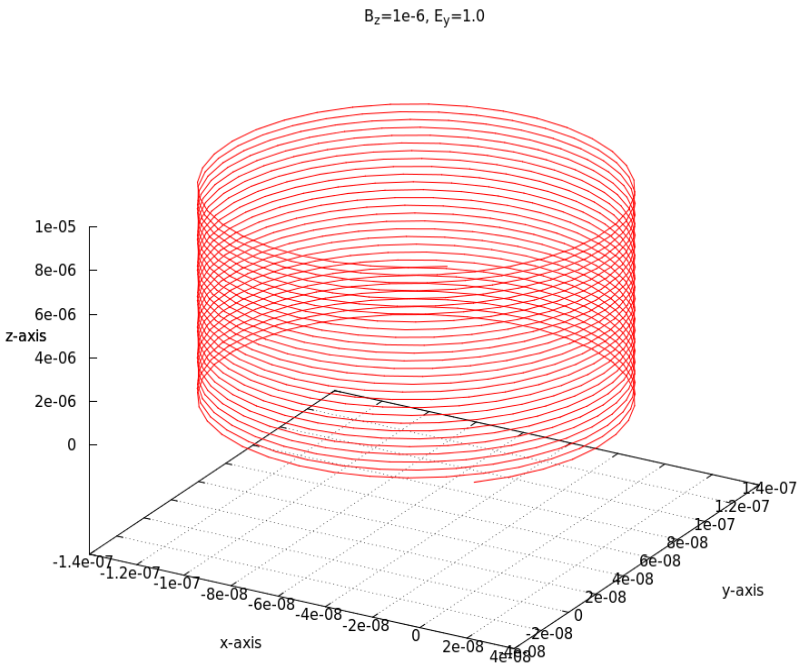


Figure 10

3.2.2 Trajectory for $B_z = 10^{-6}$ *and* $E_y = 1$

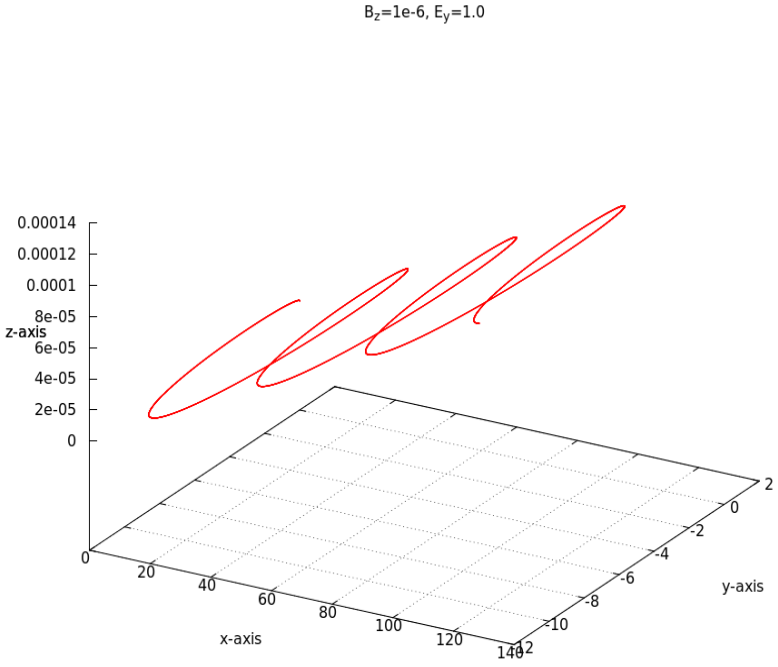


Figure 11

Following are the observations:

- For $B_z = 10^{-4}$ *and* $E = 0$ the electron moves in an accurate hellical path. (Fig. 10)
- For $B_z = 10^{-6}$ *and* $E_y = 1$ electron moves in complex cyclic forward motion due to the high electric field. (Fig. 11)

3.3 Discussion

The plots were made for a very small time interval, of order $10^{-4}s$*. Since for a electron the charge per unit mass is very high (*$1.7588\text{x}10^{11}$*), even a fairly small magnetic or electric field can give very large acceleration. Hence even times of order of* $1sec$ *are high enough for* e^- *to cover large distances (provided field is present).*

The Lorentz Force (in x and y direction) and Coulomb Force (in y direction) were vectorically added and integrated computationally to obtain the path. Also a 3D plot was used to display the electron's trajectory under electromagnetic fields in this problem.

4 Problem 4: Random walk

A drunkard takes a random walk from the origin, randomly moving one step left or right. Using the rand() function in C, simulate the motion of a one dimensional random walker. i) Find the average, and root mean square distance traveled by the random walker in time t. ii) Repeat the same as i) in 2-dimensions.

4.1 Program Code:

Note: *It has been assumed that the drunkard takes 1 step in 1 second.*

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#define M 1000 //Total no. of steps

int main()
{
    int i=0,t=0,st[2*M+1],ct[2*M+1],cty[2*M+1],x,y;
    float rmd=0.0;
    srand(time(NULL));
    FILE *f;

    //1D Motion
    f=fopen("1D.txt","wb");
    for(i=0;i<2*M+1;i++)
    {
        st[i]=i-M;
        ct[i]=0;
    }
    for(i=0;i<100*M;i++)
    {
        x=0;
        for(t=0;t<M;t++)
        { //1 second = 1 step
            if(rand()%2)
                x+=1;
            else
                x+=-1;
        }
        ++ct[x+M];
    }
    for(i=0;i<2*M+1;i+=2) //Only even no. of steps are possible!
    {
        fprintf(f,"%d\t%G\n",st[i],(1.0*ct[i])/(100.0*M));
        rmd+=ct[i]*st[i]*st[i];
    }
    rmd=sqrt(rmd/(100.0*M));
    printf("RMSD (1D) for 1000 steps : %G\n",rmd);
    fclose(f);

    //2D Motion
```

```
    f=fopen("2D.txt","wb");
    for(i=0;i<2*M+1;i++)
    {
        st[i]=i-M;
        ct[i]=0; cty[i]=0;
    }
    for(i=0;i<100*M;i++)
    {
        x=0; y=0;
        for(t=0;t<M;t++)
        { //1 second = 1 step
            switch(rand()%4)
            {
                case 0:
                    x+=1;
                break;
                case 1:
                    y+=1;
                break;
                case 2:
                    x-=1;
                break;
                case 3:
                    y-=1;
                break;
            }
        }
        ++ct[x+M]; ++cty[y+M];
    }
    rmd=0;
    for(i=0;i<2*M+1;i++)
        rmd+= (ct[i]+cty[i])*st[i]*st[i];
    rmd=sqrt(rmd/(100.0*M));
    printf("\nRMSD for 1000 steps (2D): %G\n",rmd);
    for(i=M-200;i<=M+200;i++)
        for(t=M-200;t<=M+200;t++)
            fprintf(f,"%d\t%G\t%G\n",st[i],st[t],(1.0*ct[i]*cty[t])/(10000.0*M*M));
    fclose(f);
    return 0;
}
```

4.2 Output:

```
[student@localhost SC12B156]$ ./prog4
RMSD for 1000 steps (1D): 31.5947
RMSD for 1000 steps (2D): 31.6164
[student@localhost SC12B156]$
```

Following are the Displacement output plots:

4.2.1 One Dimension

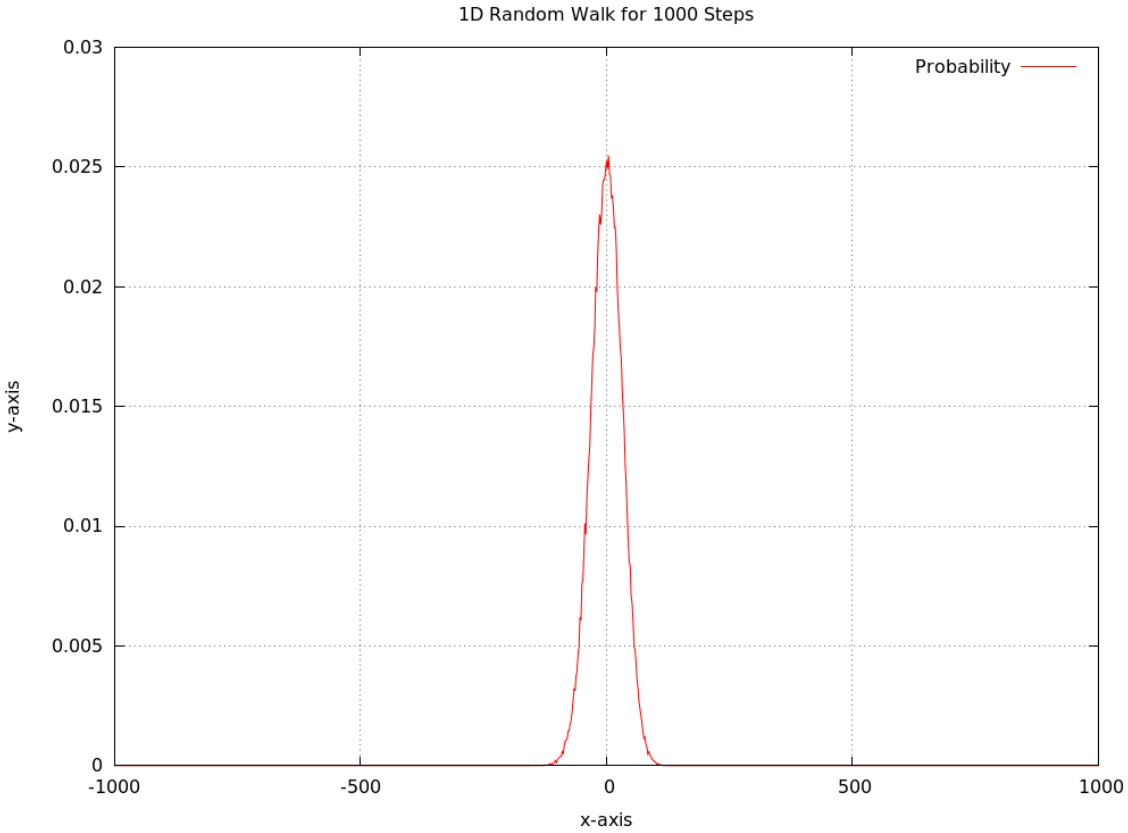


Figure 12: Probability Distribution for 1D Random Walk (RMSD=31.6)

4.2.2 Two Dimensions

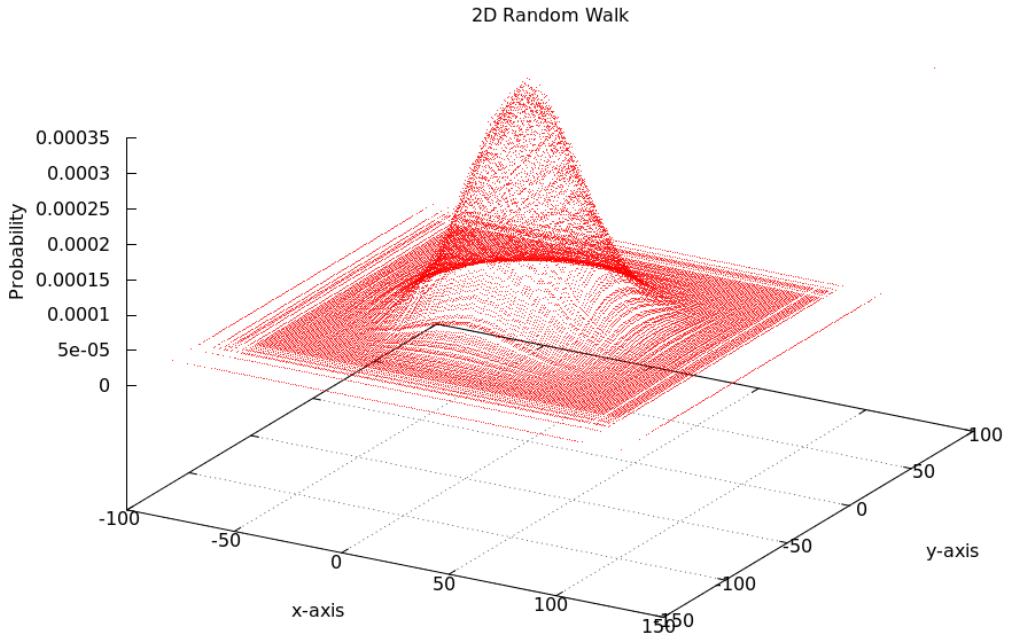


Figure 13: Probability Distribution for 2D Random Walk of 1000 steps (RMSD=31.6)

Following are the observations:

- The probability distribution of displacement form origin is a Gaussian profile with peak at 0 for both cases.
- The square mean distance is approximately $\sqrt{n} = 31.6$ (n is the step size) for both cases.

4.3 Discussion

For Random walk with equal probilities in both direction, statistics dictates that a 0 displacement is the most likely scenario. Further as n is sufficiently large, like all distributions, the displacemnt here also follows a Normal Distribution, (i.e. N(0,$\sqrt{n}$) for the 1D case. We also observe the 2D case is similar (but not strictly speaking the same) to finding the displacement profile for both coordinates individually and then performing a cross product. This point is another evidence for the cyllindrically symmetric profile in the 2D case. Of course this is expected as there is no preferred direction of axes.

That exercise does not tell us anything about how far from the orign we are after N steps! Hence we find the average positive distance away from 0 after N steps (technically the "root-mean-squared" distance) which is nothing but the Standard Deviation from the mean (0). The RMSD was statistically proven to be $\sqrt{n}$ (n is the step size) in both cases by computation. It should be noted that ***rand()*** *in C is a poor* ***pseudo-random number generator*** *and should be avoided if possible.*

Random walks can help us explain lot of natural physical phenomena like Brownian Motion, population genetics, the extraordinarily large amount of time a photon takes to emerge to the surface of a star from its core and so on.